

Part 3 — Supervised modeling and probabilities

Complete tutorial for sciModelFlowR (version 1.0.2)

Walter Esfrain Pereira

2026-09-19

Table of contents

1	From the question to the model	3
2	Diagnostics before believing	5
3	Is complexity justified?	7
4	Probabilities: classification and the distribution API	9
5	Part 3 exercises	14

1 From the question to the model

The simulated field trial has a declared truth: yield responds to nitrogen (25 kg/ha per kg/ha), potassium (6), water (3) and organic matter (120 per percentage point), plus block and genotype effects. An analyst who only has the obvious doses (nitrogen and water) starts with a **lean model**; the full model comes later, with justification.

```
field <- ler("field_yield")
v <- verdade$field_yield
no_out <- field[!field$plot_id %in% v$outlier_ids, ]
cat("plots used:", nrow(no_out), "(5 outliers excluded)\n")
```

plots used: 175 (5 outliers excluded)

```
fit_reg <- function(predictors, categorical = character(), family = "linear_regression",
                    data = no_out) {
  spec <- smf_experiment_spec(
    task = smf_task_spec("regression", "yield"),
    data = smf_data_spec("yield", predictors, id_column = "plot_id",
                        categorical = categorical),
    design = smf_design_spec(id_column = "plot_id"),
    preprocessing = smf_preprocess_spec("median"),
    resampling = smf_resampling_spec("holdout", train_prop = 0.7, seed = 21L),
    model = smf_model_spec(family, "stats"),
    metrics = list(smf_metric_spec("rmse"), smf_metric_spec("rsq")),
    reproducibility = smf_reproducibility_spec(21L)
  )
  smf_fit_experiment(spec, data)
}
lean <- fit_reg(c("nitrogen", "water"))
full <- fit_reg(c("nitrogen", "potassium", "water", "organic_matter",
                "block", "genotype"), categorical = c("block", "genotype"))
metrics_tbl <- data.frame(
  model = c("lean (N + water)", "full (N + K + water + OM + block + genotype)"),
  rmse = c(lean@metrics$value[1], full@metrics$value[1]),
  rsq = c(lean@metrics$value[2], full@metrics$value[2])
)
tab_smf(metrics_tbl, caption = "Validation performance: the full model lowers RMSE and explains")
```

Table 1.1: Validation performance: the full model lowers RMSE and explains more variance.

model	rmse	rsq
lean (N + water)	494.071	0.836
full (N + K + water + OM + block + genotype)	463.714	0.855

```
coef_full <- coef(smf_backend_object(full))
coef_tbl <- data.frame(
  parameter = c("nitrogen", "potassium", "water", "organic_matter"),
  truth = c(v$beta[["nitrogen"]], v$beta[["potassium"]],
            v$beta[["water"]], v$beta[["organic_matter"]]),
  estimated = as.numeric(coef_full[c("nitrogen", "potassium", "water", "organic_matter")])
)
coef_tbl$error <- coef_tbl$estimated - coef_tbl$truth
tab_smf(coef_tbl, caption = "Full-model coefficients against the planted truth (block and genotype effects omitted from the table).")
```

Table 1.2: Full-model coefficients against the planted truth (block and genotype effects omitted from the table).

parameter	truth	estimated	error
nitrogen	25	24.609	-0.391
potassium	6	8.139	2.139
water	3	2.953	-0.047
organic_matter	120	113.644	-6.356

i Interpretation

The full model recovers the four coefficients with an error of 6.4 kg/ha or less, and validation RMSE falls from 494 to 464. The difference between the two models is **not** “complexity”: it is the correct structure of the data. The package makes clear what entered the fit: `smf_backend_object()` returns the native model, and `fit@fit@predictors` lists the predictors used.

2 Diagnostics before believing

A model that is good on average can be wrong in regions. `smf_diagnose()` checks assumptions and returns residuals, fitted values and the list of checks.

```
diag <- smf_diagnose(full)
diag_tbl <- data.frame(
  check = c("observations evaluated", "all predictions finite",
            "missing predictions"),
  result = c(diag@checks$n, diag@checks$finite_predictions,
            diag@checks$missing_predictions)
)
tab_smf(diag_tbl, caption = "Diagnostic checks for the full model.")
```

Table 2.1: Diagnostic checks for the full model.

check	result
observations evaluated	53
all predictions finite	1
missing predictions	0

```
res_df <- data.frame(fitted = diag@fitted, resid = diag@residuals,
                    nitrogen = no_out$nitrogen[full@split@test_index])
p1 <- ggplot(res_df, aes(fitted, resid)) +
  geom_point(colour = cores_smf[["primaria"]], alpha = .7) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(x = "fitted (kg/ha)", y = "residual (kg/ha)",
       title = "Residuals vs fitted")
p2 <- ggplot(res_df, aes(nitrogen, abs(resid))) +
  geom_point(colour = cores_smf[["destaque"]], alpha = .7) +
  geom_smooth(method = "loess", formula = y ~ x, se = FALSE,
             colour = cores_smf[["alerta"]]) +
  labs(x = "nitrogen (kg/ha)", y = "|residual| (kg/ha)",
       title = "|Residuals| vs nitrogen")
patchwork::wrap_plots(p1, p2)
```

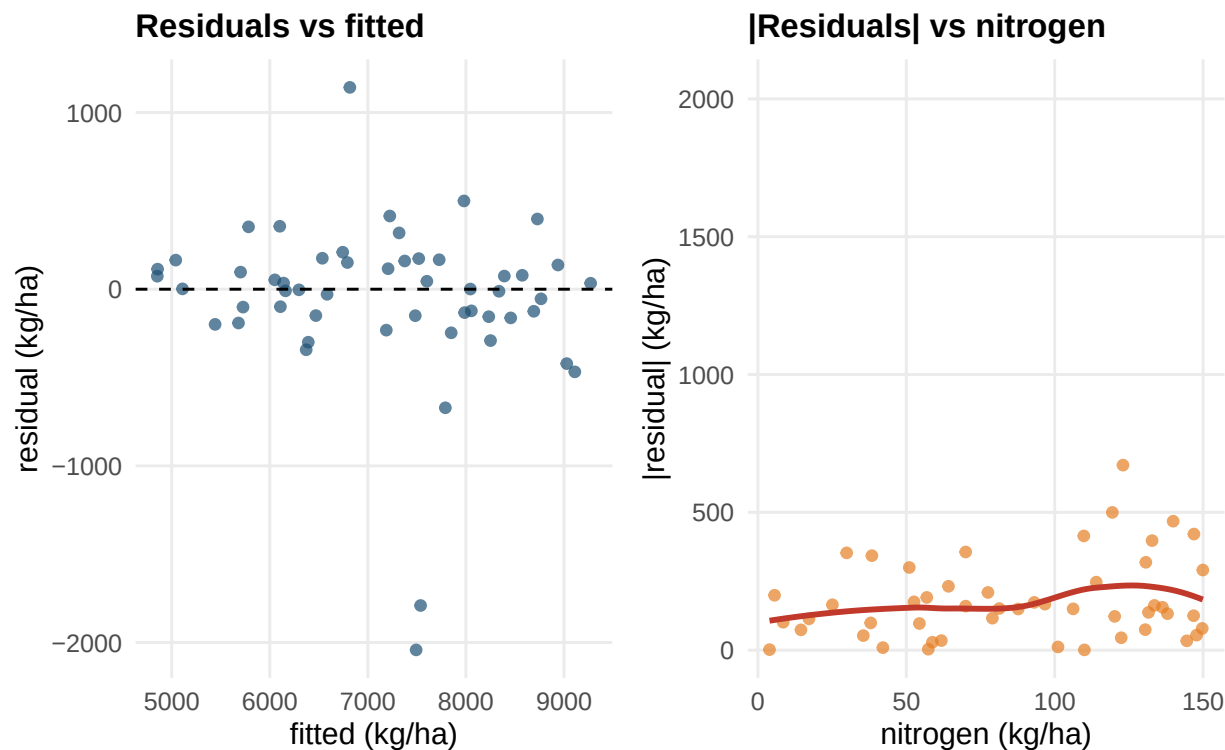


Figure 2.1: Residuals against fitted values and residual magnitude against nitrogen: spread grows with the dose — the planted heteroscedasticity shows up.

! Interpretation

Residuals centre on zero (no gross bias), but their **magnitude grows with the dose**: this is exactly the planted model, $\sigma = 100 + 1.5 * \text{nitrogen}$. Practical consequence: intervals and tests that assume constant variance will be optimistic at high doses. Parts 5 and 6 deal with this (bootstrap, conformal, uncertainty models); for now, record the diagnostic.

3 Is complexity justified?

With the same holdout and the same specification, six model families. If flexibility does not pay, the answer is clear.

```
fit_family <- function(family, engine) {
  spec <- smf_experiment_spec(
    task = smf_task_spec("regression", "yield"),
    data = smf_data_spec("yield", c("nitrogen", "potassium", "water", "organic_matter",
                                     "block", "genotype"),
                        id_column = "plot_id", categorical = c("block", "genotype")),
    design = smf_design_spec(id_column = "plot_id"),
    preprocessing = smf_preprocess_spec("median"),
    resampling = smf_resampling_spec("holdout", train_prop = 0.7, seed = 21L),
    model = smf_model_spec(family, engine),
    metrics = list(smf_metric_spec("rmse"), smf_metric_spec("rsq")),
    reproducibility = smf_reproducibility_spec(21L)
  )
  smf_fit_experiment(spec, no_out)
}
families <- data.frame(
  family = c("linear_regression", "decision_tree", "random_forest",
            "boosted_tree", "svm_rbf", "nearest_neighbor"),
  engine = c("stats", "tidymodels", "tidymodels", "tidymodels", "tidymodels", "tidymodels")
)
family_tbl <- do.call(rbind, lapply(seq_len(nrow(families)), function(i) {
  f <- fit_family(families$family[i], families$engine[i])
  data.frame(family = families$family[i],
            rmse = f@metrics$value[f@metrics$metric == "rmse"],
            rsq = f@metrics$value[f@metrics$metric == "rsq"])
}))
family_tbl <- family_tbl[order(family_tbl$rmse), ]
tab_smf(family_tbl, caption = "Six families on the same holdout: validation RMSE and R-squared")
```

Table 3.1: Six families on the same holdout: validation RMSE and R-squared.

	family	rmse	rsq
1	linear_regression	463.714	0.855
5	svm_rbf	520.777	0.818
2	decision_tree	547.112	0.799
4	boosted_tree	590.740	0.765

	family	rmse	rsq
3	random_forest	624.697	0.738
6	nearest_neighbor	1066.772	0.235

```
ggplot(family_tbl, aes(stats::reorder(family, -rmse), rmse)) +
  geom_col(fill = cores_smf[["primaria"]], width = .6) +
  geom_text(aes(label = fmt(rmse, 0)), vjust = -0.4) +
  labs(x = NULL, y = "validation RMSE (kg/ha)",
       title = "Model families on the same holdout") +
  ylim(0, max(family_tbl$rmse) * 1.12)
```

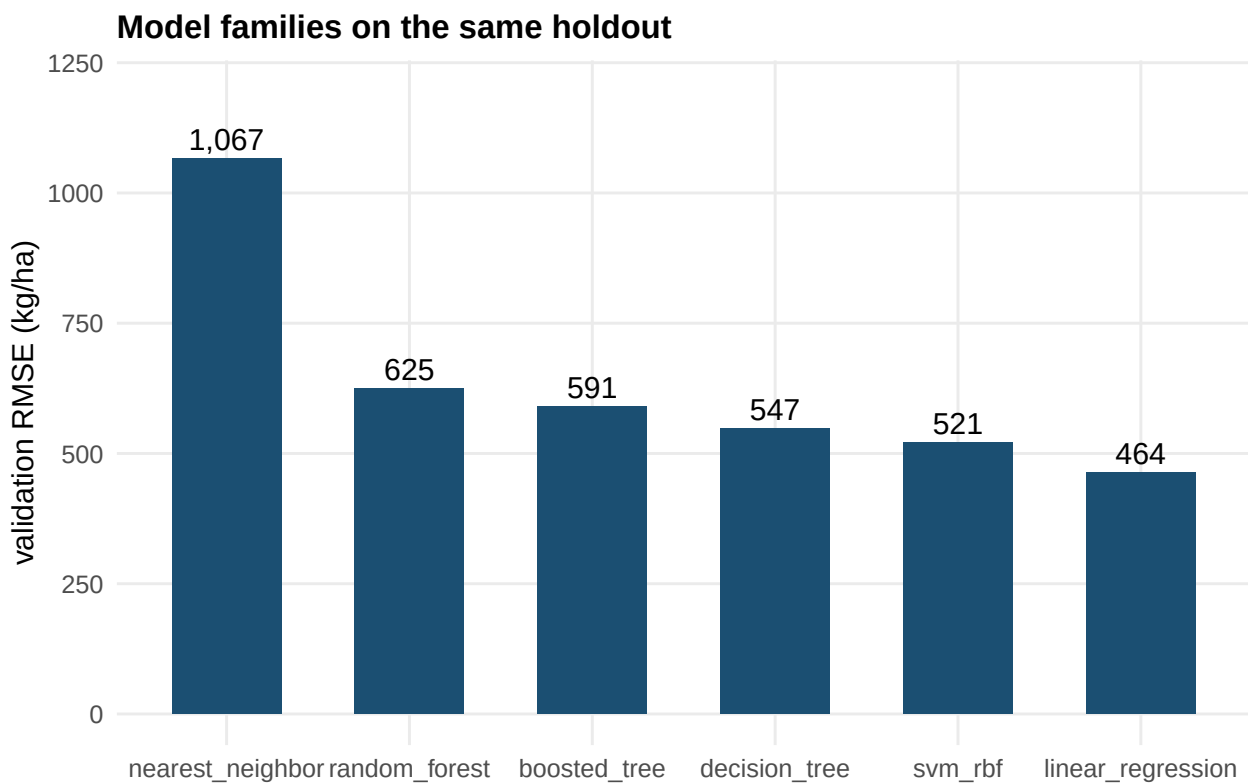


Figure 3.1: Family comparison: the straight line recovers the truth; trees and nearest neighbours do not beat the correct linear structure.

⚠ Trap

The temptation is always “use the strongest model”. Here, the **linear** model wins because the data were generated linear — and no flexible family compensates for the extra variance. In other datasets the answer flips (we will see it in classification, just below). The criterion is not the prestige of the method: it is performance on new observations, measured in the right design.

4 Probabilities: classification and the distribution API

In the disease dataset, the outcome is binary, imbalanced and has 5% flipped labels — and the true boundary has a quadratic term. Compare logistic regression with a random forest.

```
disease <- ler("disease_binary")
disease$disease <- factor(disease$disease, levels = c(0, 1))
fit_cls <- function(family, engine) {
  spec <- smf_experiment_spec(
    task = smf_task_spec("binary", "disease", positive_label = "1"),
    data = smf_data_spec("disease", c("x1", "x2", "x3", "x4"), id_column = "plant_id"),
    design = smf_design_spec(id_column = "plant_id"),
    preprocessing = smf_preprocess_spec("median", center = TRUE, scale = TRUE),
    resampling = smf_resampling_spec("stratified_holdout", train_prop = 0.75,
                                     strata = "disease", seed = 22L),
    model = smf_model_spec(family, engine),
    metrics = list(smf_metric_spec("log_loss"), smf_metric_spec("brier"),
                  smf_metric_spec("roc_auc")),
    reproducibility = smf_reproducibility_spec(22L)
  )
  smf_fit_experiment(spec, disease)
}
logit <- fit_cls("logistic_regression", "stats")
forest <- fit_cls("random_forest", "tidymodels")
cls_tbl <- rbind(
  cbind(model = "logistic", logit@metrics[, c("metric", "value")]),
  cbind(model = "random forest", forest@metrics[, c("metric", "value")])
)
tab_smf(cls_tbl, caption = "Disease classification: the forest captures the quadratic term th
```

Table 4.1: Disease classification: the forest captures the quadratic term that logistic regression cannot see.

model	metric	value
logistic	log_loss	0.379
logistic	brier	0.221
logistic	roc_auc	0.652
random forest	log_loss	0.610
random forest	brier	0.180
random forest	roc_auc	0.804

model	metric	value
-------	--------	-------

```
test <- disease[forest@split@test_index, ]
p_rf <- forest@prediction@probabilities[, "1"]
p_lg <- logit@prediction@probabilities[, "1"]
curve_roc <- function(p, y) {
  o <- order(p, decreasing = TRUE)
  y <- as.integer(y[o] == "1")
  data.frame(fpr = cumsum(1 - y) / sum(1 - y),
             tpr = cumsum(y) / sum(y))
}
roc <- rbind(cbind(model = "logistic", curve_roc(p_lg, test$disease)),
            cbind(model = "random forest", curve_roc(p_rf, test$disease)))
```

```
ggplot(roc, aes(fpr, tpr, colour = model)) +
  geom_line(linewidth = 1) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour = "grey60") +
  scale_colour_manual(values = c(logistic = cores_smf[["neutro"]],
                                `random forest` = cores_smf[["primaria"]])) +
  coord_equal() +
  labs(x = "false positives (1 - specificity)", y = "true positives (sensitivity)",
       colour = NULL,
       title = "ROC curves on the test partition",
       subtitle = paste0("AUC logistic = ",
                          fmt(logit@metrics$value[logit@metrics$metric == "roc_auc"], 3),
                          " | forest = ",
                          fmt(forest@metrics$value[forest@metrics$metric == "roc_auc"], 3),
                          " | oracle ceiling = 0.84")) +
  theme(legend.position = "top")
```

ROC curves on the test partition

AUC logistic = 0.652 | forest = 0.804 | oracle ceiling = 0.84

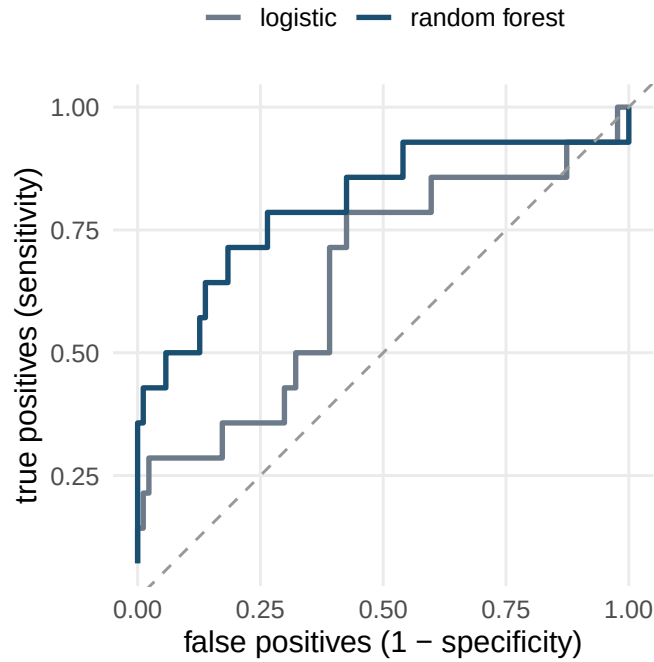


Figure 4.1: ROC curves on the test partition: the forest approaches the ceiling imposed by label noise (oracle AUC = 0.84).

A probabilistic prediction is a distribution object. The package has its own API to operate these distributions — mean, quantiles, log-probability, intervals — and to score them.

```
pd_rf <- smf_prediction_distribution(  
  "class_probabilities",  
  list(prob = forest@prediction@probabilities, classes = c("0", "1"))  
)  
scores_rf <- smf_evaluate_probabilistic(test$disease, pd_rf,  
                                       c("log_loss", "brier", "ece"))  
tab_smf(scores_rf, caption = "Probabilistic scores of the forest (ECE: expected calibration error)")
```

Table 4.2: Probabilistic scores of the forest (ECE: expected calibration error).

metric	value	direction	n
log_loss	0.610	minimize	101
brier	0.180	minimize	101
ece	0.081	minimize	101

```
normal <- smf_prediction_distribution(  
  "parametric",
```

```

  list(distribution = "normal", mean = c(5000, 6000), sd = c(300, 450))
)
dist_tbl <- data.frame(
  operation = c("mean", "5% quantile", "95% quantile", "log-probability at 5000"),
  result = c(paste(fmt(smf_dist_mean(normal), 0), collapse = " | "),
    paste(fmt(smf_dist_quantile(normal, 0.05), 0), collapse = " | "),
    paste(fmt(smf_dist_quantile(normal, 0.95), 0), collapse = " | "),
    paste(fmt(smf_dist_log_prob(normal, c(5000, 5000)), 3), collapse = " | "))
)
tab_smf(dist_tbl, caption = "Operations on predictive distributions: two hypothetical plots w

```

Table 4.3: Operations on predictive distributions: two hypothetical plots with means 5000 and 6000 kg/ha.

operation	result
mean	5,000 6,000
5% quantile	4,507 5,260
95% quantile	5,493 6,740
log-probability at 5000	-6.623 -9.497

```

prob_df <- data.frame(p = p_rf, class = test$disease)
ggplot(prob_df, aes(p, fill = class)) +
  geom_histogram(position = "identity", alpha = .55, bins = 25, colour = "white") +
  scale_fill_manual(values = c(`0` = cores_smf[["claro"]], `1` = cores_smf[["alerta"]]),
    labels = c("healthy (0)", "diseased (1)")) +
  labs(x = "predicted probability of disease", y = "plants", fill = NULL,
    title = "Predicted probabilities by true class") +
  theme(legend.position = "top")

```

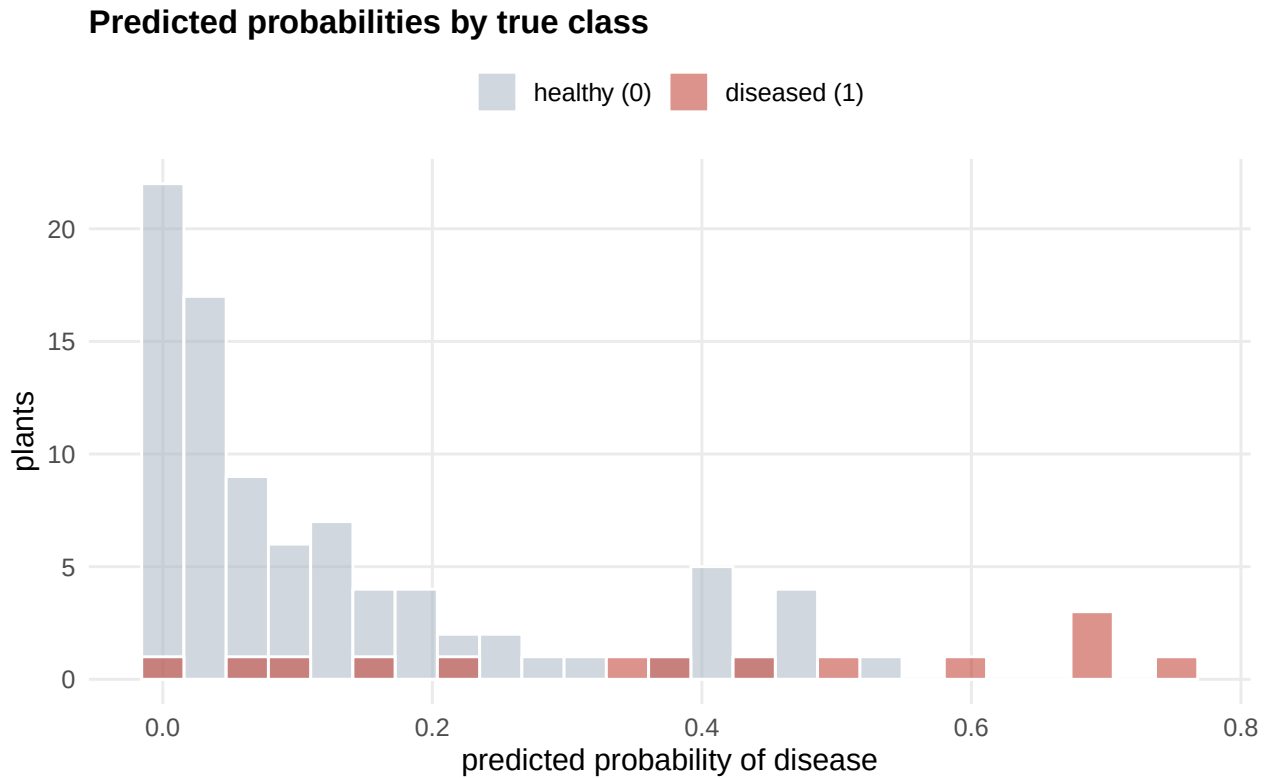


Figure 4.2: Distribution of predicted probabilities by true class: the overlap is label noise in action.

i Interpretation

The forest nearly doubles the separation of logistic regression (AUC 0.80 against 0.65), but no model can reach 1.0: label noise imposes a ceiling (oracle AUC = 0.84) and the probability distributions overlap. An honest classifier communicates **probability**, not certainty — and calibrating those probabilities is the theme of Part 4.

5 Part 3 exercises

Exercise 3.1 — The price of five points

Refit the full model **with** the five outliers and compare RMSE, R-squared and the organic-matter coefficient with those of the outlier-free model. How many points changed the conclusion?

Exercise 3.2 — Giving eyes to logistic regression

Create the column `x3_sq = x3^2` in the disease dataset and repeat the logistic regression with `x1`, `x2`, `x3`, `x3_sq`, `x4`. Does AUC approach the oracle (0.84)? What does this say about “simple versus complex model”?

Exercise 3.3 — Accuracy deceives

With threshold 0.5, compute the accuracy of a classifier that predicts “healthy” for every plant. Compare with the forest accuracy on the test set and explain the difference in terms of prevalence.

Exercise 3.4 — Threshold sweep

For the forest, compute sensitivity and specificity at thresholds from 0.1 to 0.9 and describe the trade-off. Where would you place the threshold for a disease control programme in which a false negative costs ten times more than a false positive?

Solutions

3.1. With the outliers, RMSE rises, R-squared falls and the organic-matter coefficient moves away from the truth (as in Part 2, it almost triples). Five points out of 180 are enough to change the conclusion — disproportionate influence.

3.2. With `x3_sq`, logistic AUC rises substantially (the quadratic term was what was missing). The same “simple” model becomes competitive: the problem was never the family, it was the **functional form**.

3.3. Predicting “healthy” for everyone gets the fraction of negatives right (about 90%), but has zero sensitivity. The forest has lower accuracy than that at many thresholds and is still infinitely more useful: accuracy does not measure what matters under imbalance.

3.4. Sensitivity grows and specificity falls as the threshold decreases. With a false negative ten times more costly, the threshold should be **low** (more plants classified as diseased), accepting more false positives. Part 4 formalises this with explicit costs.